

JDBC + Servlets + JSP — Product List Assignment Answers

Topics: JDBC connection, ResultSet mapping, SQL exception handling, LIKE search

Q1. ProductListServlet — Connecting to MySQL via JDBC

The products table is assumed to have columns: id (INT), name (VARCHAR), price (DECIMAL/DOUBLE). The servlet uses the MySQL JDBC driver (mysql-connector-j) to open a connection and run a query.

```
private static final String DB_URL = "jdbc:mysql://localhost:3306/shopdb";
private static final String DB_USER = "root";
private static final String DB_PASS = "yourpassword";

protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    List<Product> productList = new ArrayList<>();

    try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS)) {
        String sql = "SELECT id, name, price FROM products";

        try (PreparedStatement stmt = conn.prepareStatement(sql);
             ResultSet rs = stmt.executeQuery()) {

            while (rs.next()) {
                Product p = new Product(rs.getInt("id"), rs.getString("name"), rs.getDouble("price"));
                productList.add(p);
            }
        }
        request.setAttribute("productList", productList);
    }

    RequestDispatcher dispatcher = request.getRequestDispatcher("products.jsp");
    dispatcher.forward(request, response);
}
```

Setup note: the MySQL JDBC driver JAR (mysql-connector-j-x.x.x.jar) must be added to the project's WEB-INF/lib folder for Class.forName/DriverManager to find the 'com.mysql.cj.jdbc.Driver'.

Q2. products.jsp — Displaying the Product List in a Table

```
<%
    List<Product> productList = (List<Product>) request.getAttribute("productList");
%>
<table border="1">
    <tr><th>ID</th><th>Name</th><th>Price</th></tr>
    <%
        for (Product p : productList) {
    %>
        <tr>
            <td><%= p.getId() %></td>
            <td><%= p.getName() %></td>
            <td><%= p.getPrice() %></td>
        </tr>
    <%
        }
    %>
</table>
```

(Full styled version with search bar and error handling: **products.jsp**, attached.)

Q3. Handling SQLException — Friendly Error on the JSP

If getConnection() or the query throws a SQLException (e.g. wrong credentials, DB server down), the servlet catches it, logs the real exception on the server console, and sets a friendly error message as a request attribute instead of letting the exception crash the page.

```
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASS)) {
    // ... query + populate productList ...
    request.setAttribute("productList", productList);

} catch (SQLException e) {
    e.printStackTrace(); // real error goes to the server log for debugging
    request.setAttribute("dbError",
        "We're having trouble connecting to the database right now. Please try again later.");
}
```

products.jsp checks for this attribute first:

```
<%
    String dbError = (String) request.getAttribute("dbError");
    if (dbError != null) {
%>
        <p style="color:red; font-weight:bold;"><%= dbError %></p>
<%
    } else {
        // ... render the product table as normal ...
    }
%>
```

Q4. Search Feature — SQL LIKE for Partial, Case-Insensitive Matching

If a 'name' request parameter is present, the SQL query switches to a LIKE clause with wildcards. Wrapping both the column and the search term in LOWER() makes the match case-insensitive; using a PreparedStatement with a '?' placeholder (rather than string concatenation) also protects against SQL injection.

```
String searchName = request.getParameter("name");
boolean isSearch = (searchName != null && !searchName.trim().isEmpty());

String sql = isSearch
    ? "SELECT id, name, price FROM products WHERE LOWER(name) LIKE ?"
    : "SELECT id, name, price FROM products";

try (PreparedStatement stmt = conn.prepareStatement(sql)) {
    if (isSearch) {
        stmt.setString(1, "%" + searchName.toLowerCase() + "%");
    }
    try (ResultSet rs = stmt.executeQuery()) {
        while (rs.next()) {
            productList.add(new Product(rs.getInt("id"), rs.getString("name"), rs.getDouble("price")));
        }
    }
}
```

Example: /ProductListServlet?name=shoe matches 'Running Shoes', 'shoe rack', 'SHOES', etc. — any product whose name contains 'shoe' regardless of case or position in the string. products.jsp includes a small search form that submits back to ProductListServlet with this 'name' parameter (see the attached products.jsp).